Practical:10	Implementation of Kruskal's algorithm using greedy method.
__/__/2026	

Objective:

- To implement Kruskal's algorithm to find the Minimum Spanning Tree (MST) of a connected, weighted, undirected graph.
- To understand how the greedy method picks the cheapest edge that does not form a cycle, and how Union-Find detects cycles.
- To analyze the time and space complexity of Kruskal's algorithm.

Program:

```
# Kruskal's Minimum Spanning Tree (edge list + Union-Find)
n = 6  # number of vertices

# each edge is stored as (weight, u, v)
edges = [
    (4, 0, 1), (3, 0, 2), (1, 1, 2), (2, 1, 3),
    (4, 2, 3), (2, 3, 4), (7, 3, 5), (6, 4, 5)
]

parent = list(range(n))
rank = [0] * n

def find(x):
    # find the root of x (with path compression)
    if parent[x] != x:
        parent[x] = find(parent[x])
    return parent[x]

def union(a, b):
    # join the sets of a and b; return False if already in the same set
    ra, rb = find(a), find(b)
    if ra == rb:
        return False
    if rank[ra] < rank[rb]:
        ra, rb = rb, ra
    parent[rb] = ra
    if rank[ra] == rank[rb]:
        rank[ra] += 1
    return True

edges.sort()  # sort edges by weight (smallest first)
mst = []
total = 0

print("KRUSKAL'S ALGORITHM")
```

```
for w, u, v in edges:
    if union(u, v):
        mst.append((u, v, w))
        total += w
        print(f"Add edge {u}-{v} (weight {w})")
    else:
        print(f"Skip edge {u}-{v} (weight {w}) : forms a cycle")
    if len(mst) == n - 1:
        break

if len(mst) != n - 1:
    raise ValueError("Graph is disconnected")

print("Edges in the MST:")
for u, v, w in mst:
    print(f"  {u} - {v} : {w}")
print("Total cost of MST =", total)
```

Output:

```
KRUSKAL'S ALGORITHM
Add edge 1-2 (weight 1)
Add edge 1-3 (weight 2)
Add edge 3-4 (weight 2)
Add edge 0-2 (weight 3)
Skip edge 0-1 (weight 4) : forms a cycle
Skip edge 2-3 (weight 4) : forms a cycle
Add edge 4-5 (weight 6)
Edges in the MST:
  1 - 2 : 1
  1 - 3 : 2
  3 - 4 : 2
  0 - 2 : 3
  4 - 5 : 6
Total cost of MST = 14
```

Analysis:

a) Problem Analysis

A spanning tree of a graph connects all V vertices using exactly V - 1 edges and has no cycle. A Minimum Spanning Tree is the spanning tree whose total edge weight is the smallest.

Kruskal's algorithm solves the same problem as Prim's algorithm (Practical 9), but it works on the list of edges instead of growing a single tree from one vertex. Typical uses are laying cables, pipelines or roads at minimum cost.

b) Greedy Approach

Kruskal's algorithm starts with every vertex as its own separate tree. It then looks at the edges in increasing order of weight and adds an edge only if it joins two different trees, that is, only if it does not form a cycle. This is safe because the cheapest edge connecting two different

components always belongs to some MST (the cut property).

To check for a cycle quickly, the program uses Union-Find (Disjoint Set Union) with path compression and union by rank. An edge u - v forms a cycle exactly when find(u) = find(v), because both ends are already in the same tree.

c) Example Analysis

Consider the same graph with 6 vertices (0 to 5) as in Practical 9. Its edges, sorted by weight, are:

Edge	Weight
1 - 2	1
1 - 3	2
3 - 4	2
0 - 2	3
0 - 1	4
2 - 3	4
4 - 5	6
3 - 5	7

Trace of the algorithm:

Step	Edge	Weight	Action	Cost so far	Components after
1	1-2	1	Add	1	{1,2} {0} {3} {4} {5}
2	1-3	2	Add	3	{1,2,3} {0} {4} {5}
3	3-4	2	Add	5	{1,2,3,4} {0} {5}
4	0-2	3	Add	8	{0,1,2,3,4} {5}
5	0-1	4	Reject (cycle)	8	no change
6	2-3	4	Reject (cycle)	8	no change
7	4-5	6	Add	14	{0,1,2,3,4,5}

Edges 0 - 1 and 2 - 3 are rejected because their end vertices are already connected. After step 7 the tree has 5 edges (V - 1), so the algorithm stops and edge 3 - 5 is never examined. Total cost of the MST = 14, the same as Prim's algorithm gave.

d) Algorithm Analysis

Step 1: Sort all edges in increasing order of weight.

Step 2: Make every vertex its own set: parent[i] = i.

Step 3: Take the next edge u - v from the sorted list.

Step 4: If find(u) is not equal to find(v), add the edge to the MST, add its weight to the total cost

and union the two sets. Otherwise skip the edge, because it would form a cycle.

Step 5: Repeat steps 3 and 4 until V - 1 edges are chosen. If the edges run out before that, the graph is disconnected.

e) Time Complexity

Sorting the E edges takes O(E log E). Each find and union call is almost constant time, $O(\alpha(V))$, when path compression and union by rank are used, so going through all the edges costs $O(E\ \alpha(V))$, which is smaller than the sorting cost.

Therefore:

Time Complexity = O(E log E) = O(E log V)

Since E can be at most V^2, log E is O(log V), so both forms are the same. Kruskal's algorithm suits sparse graphs well, while Prim's with an adjacency matrix runs in $O(V^2)$.

f) Space Complexity

The edge list needs O(E) space. The parent and rank arrays need O(V) space.

Therefore:

Space Complexity = O(E + V)

Conclusion:

Kruskal's algorithm was successfully implemented in Python using the greedy method and Union-Find. For the given graph it selected 5 edges (V - 1), rejected the edges 0 - 1 and 2 - 3 because they would form cycles, and gave a total MST cost of 14, the same as Prim's algorithm. The running time is O(E log E), which is dominated by sorting the edges.